
B28: SQL SuperHero Program : Session 17

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

12 March 2025 at 01:19

WINDOW FUNCTION

```
CREATE TABLE SALES
(
  SALE_ID    INTEGER,
  PRODUCT_ID INTEGER,
  YEAR       INTEGER,
  Quantity   INTEGER,
  PRICE      INTEGER
);

INSERT INTO SALES VALUES ( 1, 100, 2008, 10, 5000);
INSERT INTO SALES VALUES ( 2, 100, 2009, 12, 5000);
INSERT INTO SALES VALUES ( 3, 100, 2010, 25, 5000);
INSERT INTO SALES VALUES ( 4, 100, 2011, 16, 5000);
INSERT INTO SALES VALUES ( 5, 100, 2012, 8, 5000);

INSERT INTO SALES VALUES ( 6, 200, 2010, 10, 9000);
INSERT INTO SALES VALUES ( 7, 200, 2011, 15, 9000);
INSERT INTO SALES VALUES ( 8, 200, 2012, 20, 9000);
INSERT INTO SALES VALUES ( 9, 200, 2008, 13, 9000);
INSERT INTO SALES VALUES ( 10, 200, 2009, 14, 9000);

INSERT INTO SALES VALUES ( 11, 300, 2010, 20, 7000);
INSERT INTO SALES VALUES ( 12, 300, 2011, 18, 7000);
INSERT INTO SALES VALUES ( 13, 300, 2012, 20, 7000);
INSERT INTO SALES VALUES ( 14, 300, 2008, 17, 7000);
INSERT INTO SALES VALUES ( 15, 300, 2009, 19, 7000);

COMMIT;
```

We have to use the option "ROWS UNBOUNDED PRECEDING" in the SUM analytic function to get the cumulative sum.

ROWS BETWEEN 1 PRECEDING AND CURRENT ROW - the previous row and the current row.
ROWS BETWEEN 3 PRECEDING AND 1 FOLLOWING - the 3 previous rows, the current row, and the following row.
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING - all rows in the partition.

--> You can create any permutation and combination of preceding and following

```
SELECT PRODUCT_ID,
  QUANTITY,
  SUM(QUANTITY) OVER(
    PARTITION BY PRODUCT_ID
    ORDER BY QUANTITY DESC
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING ) CALC_SALES
FROM SALES;
```

```

8 SELECT PRODUCT_ID,
9    QUANTITY,
10   SUM(QUANTITY) OVER(
11     PARTITION BY PRODUCT_ID
12     ORDER BY QUANTITY DESC
13     ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING ) CALC_SALES
14 FROM SALES;

```

all records

	PRODUCT_ID	QUANTITY	CALC_SALES
1	100	25	71
2	100	16	71
3	100	12	71
4	100	10	71
5	100	8	71
6	200	20	72
7	200	15	72
8	200	14	72
9	200	13	72
10	200	10	72
11	300	20	94
12	300	20	94
13	300	19	94
14	300	18	94

$25 + 16 + 12 + 10 + 8 = 71$

```

SELECT PRODUCT_ID,
QUANTITY,
SUM(QUANTITY) OVER(
PARTITION BY PRODUCT_ID
ORDER BY QUANTITY DESC
ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING ) CALC_SALES
FROM SALES;

```

--> Here in above query, we are doing calculative sum, bu using window clause ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING

--> It means, 2 rows before + current row + 2 row after that

within boundary

```

7
8 SELECT PRODUCT_ID,
9    QUANTITY,
10   SUM(QUANTITY) OVER(
11     PARTITION BY PRODUCT_ID
12     ORDER BY QUANTITY DESC
13     ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING ) CALC_SALES
14 FROM SALES;

```

2 Rows Before + current + 2 Rows after

	PRODUCT_ID	QUANTITY	CALC_SALES
1	100	25	53
2	100	16	63
3	100	12	71
4	100	10	46
5	100	8	30
6	200	20	49
7	200	15	62
8	200	14	72
9	200	13	52
10	200	10	37
11	300	20	59
12	300	20	77
13	300	19	94
14	300	18	74

$25 + 16 + 12 = 53$

$25 + 16 + 12 + 10 = 63$

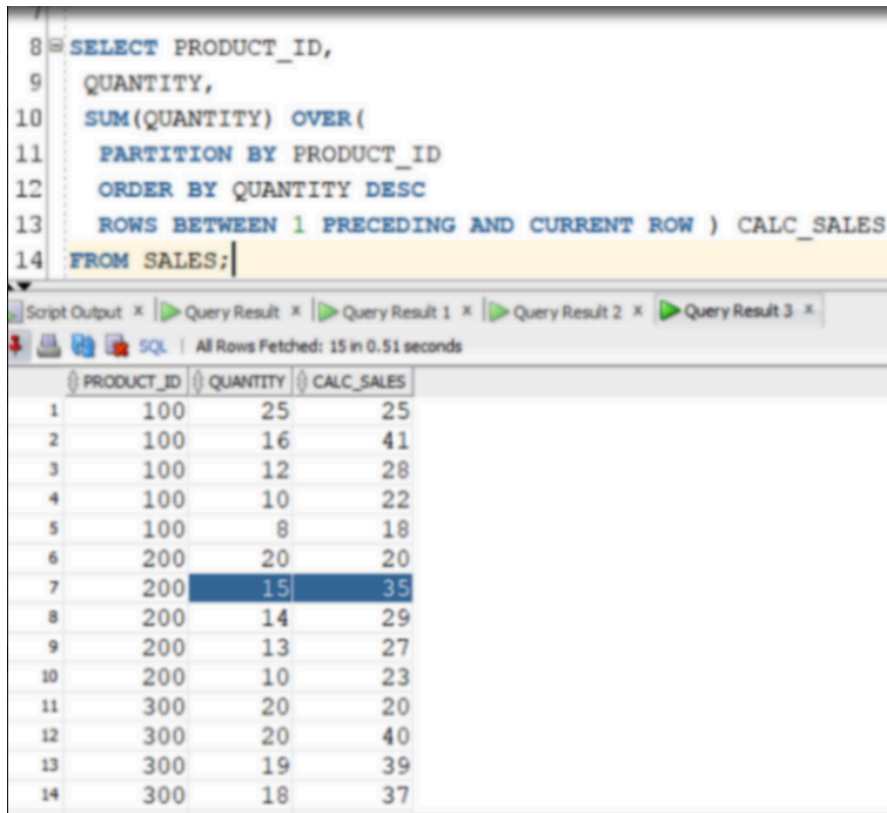
2 following

*for 8 -> No following
beoz Product id
changes*

--> for product_id = 100 is one boundary , 200 is another and 300 is another, because we are doing partition by product_id
--> so all analytical function will play the role within boundry.

for 8, rownumber = 5, we have 2 preceding but no following, because the next record is for product_id = 200,
so boundary is changed, and this function works in boundary only.

--> for 20, rownumber = 6, there is no preceding, because previous rows are from another product_id and here we
are doing partition by product_id, so it only took, current and 2 following



```
8 SELECT PRODUCT_ID,  
9    QUANTITY,  
10   SUM(QUANTITY) OVER(  
11     PARTITION BY PRODUCT_ID  
12     ORDER BY QUANTITY DESC  
13     ROWS BETWEEN 1 PRECEDING AND CURRENT ROW ) CALC_SALES  
14 FROM SALES;
```

	PRODUCT_ID	QUANTITY	CALC_SALES
1	100	25	25
2	100	16	41
3	100	12	28
4	100	10	22
5	100	8	18
6	200	20	20
7	200	15	35
8	200	14	29
9	200	13	27
10	200	10	23
11	300	20	20
12	300	20	40
13	300	19	39
14	300	18	37

```
SELECT PRODUCT_ID,year,  
QUANTITY,  
SUM(QUANTITY) OVER(  
PARTITION BY year  
ORDER BY QUANTITY DESC  
ROWS BETWEEN 1 PRECEDING AND 2 FOLLOWING ) CALC_SALES  
FROM SALES;
```

```

8 SELECT PRODUCT_ID, year,
9    QUANTITY,
10   SUM(QUANTITY) OVER(
11     PARTITION BY year
12     ORDER BY QUANTITY DESC
13     ROWS BETWEEN 1 PRECEDING AND 2 FOLLOWING ) CALC_SALES
14 FROM SALES;

```

Script Output x Query Result x Query Result 1 x Query Result 2 x Query Result 3 x Query Result 4 x

All Rows Fetched: 15 in 0.525 seconds

	PRODUCT_ID	YEAR	QUANTITY	CALC_SALES
1	300	2008	17	40
2	200	2008	13	40
3	100	2008	10	23
4	300	2009	19	45
5	200	2009	14	45
6	100	2009	12	26
7	100	2010	25	55
8	300	2010	20	55
9	200	2010	10	30
10	300	2011	18	49
11	100	2011	16	49
12	200	2011	15	31
13	300	2012	20	48
14	200	2012	20	48

Handwritten notes:

- Red arrow: No Preceding $17 + 13 + 10 = 40$
- Blue arrow: $17 + 13 + 10 = 40$

Handwritten note: All Previous + current

```

select S.*, sum(quantity) over(partition by year order by quantity desc rows between unbounded preceding and current row) xxx from sales

```

Handwritten note: one change start again

SALE_ID	PRODUCT_ID	YEAR	QUANTITY	PRICE	XXX
14	300	2008	17	7000	17
9	200	2008	13	9000	30
1	100	2008	10	5000	40
15	300	2009	19	7000	19
10	200	2009	14	9000	33
2	100	2009	12	5000	45
3	100	2010	25	5000	25
11	300	2010	20	7000	45
6	200	2010	10	9000	55
12	300	2011	18	7000	18
4	100	2011	16	5000	34
7	200	2011	15	9000	40

PROBLEM STATEMENT

LEVEL 2

The maximum weight the elevator can hold is 1000.

Write an SQL query to find the name of the last person who will fit in the elevator without exceeding the weight limit. It is guaranteed that the person who is first in the queue can fit in the elevator.

TABLE : QUEUE

person_id	person_name	weight	turn
5	George W	250	1
6	Thomas J	400	3
3	John A	350	2
4	Thomas R	175	5
2	Will J	200	4
1	James E	500	6

Read this question. This table QUEUE has a column TURN , means people are going inside the elevator as per their turns. so 1st George will enter and then John and then Thomas then Will then Thomas R and then James.
Question is saying, that elevator can maximum take weight of 1000. It means we have to sum the weight of the next person and as soon as we reach 1000, we have to stop.

```
CREATE TABLE QUEUE_T
(
  P_ID  NUMBER,
  P_NAME VARCHAR2(50 BYTE),
  P_W   NUMBER,
  P_TURN NUMBER
);
```

```
Insert into QUEUE_T (P_ID, P_NAME, P_W, P_TURN) Values (5, 'George W', 250, 1);
Insert into QUEUE_T (P_ID, P_NAME, P_W, P_TURN) Values (6, 'Thomas J', 400, 3);
Insert into QUEUE_T (P_ID, P_NAME, P_W, P_TURN) Values (3, 'John A', 350, 2);
Insert into QUEUE_T (P_ID, P_NAME, P_W, P_TURN) Values (4, 'Thomas R', 175, 5);
Insert into QUEUE_T (P_ID, P_NAME, P_W, P_TURN) Values (2, 'Will J', 200, 4);
Insert into QUEUE_T (P_ID, P_NAME, P_W, P_TURN) Values (1, 'James E', 500, 6);
```

Worksheet

Query Builder

```
select * from QUEUE_T
```

 Query Result ✕

 SQL | All Rows Fetched: 6 in 0.003 seconds

	P_ID	P_NAME	P_W	P_TURN
1	5	George W	250	1
2	6	Thomas J	400	3
3	3	John A	350	2
4	4	Thomas R	175	5
5	2	Will J	200	4
6	1	James E	500	6

`select T.*, sum(P_W) over(order by P_TURN rows between unbounded PRECEDING and current row) from QUEUE_T T ;`

select T., sum(P_W) over(order by P_TURN rows between unbounded preceding and current row) xx from QUEUE_T T*

P_ID	P_NAME	P_W	P_TURN	XX
5	George W	250	1	250
3	John A	350	2	600
6	Thomas J	400	3	1000
2	Will J	200	4	1200
4	Thomas R	175	5	1375
1	James E	500	6	1875

6 rows selected.

Handwritten notes:
 - *All previous + current* (pointing to the window definition)
 - *Arrange data by turn* (pointing to the ORDER BY clause)
 - *250 + 350*
600 + 400
 - *cumulative sum* (pointing to the XX column)

If you see the above output, check the column xx which is populated via the concept of unbounded preceding and current row. means cumulative sum is population. Now we want to find out who is the last person who enter in elevator. so we have to check when it reached 1000.

```
with vishal as (
select T.*, sum(P_W) over(order by P_TURN rows between unbounded preceding and current row) xx from QUEUE_T T
) select * from vishal
```

with clause used to see the whole data in a memory

P_ID	P_NAME	P_W	P_TURN	XX
5	George W	250	1	250
3	John A	350	2	600
6	Thomas J	400	3	1000
2	Will J	200	4	1200
4	Thomas R	175	5	1375
1	James E	500	6	1875

6 rows selected.

Worksheet Query Builder

```
with vishal as (select T.*, sum(P_W) over(order by t.p_turn rows between unbounded preceding and current row ) xx from queue_T T)
select * from vishal where xx = 1000
```

Just add this where condition

Query Result x

SQL | All Rows Fetched: 1 in 0.002 seconds

P_ID	P_NAME	P_W	P_TURN	XX
6	Thomas J	400	3	1000

Answer

=====

--> Lets change the question: If I want to adjust as many people as I have too. Means, forget about turn, I will start entering with lowest weight .

Worksheet Query Builder

```
1 with n1 as (select t.*, sum(p_w) over(order by p_w rows between unbounded preceding and current row) xx
2 from queue_tt)
3 select n1.* from n1
4
```

Arrange data by weight

Script Output x Query Result x Query R... x

SQL | All Rows Fetched: 6 in 0.043 seconds

P_ID	P_NAME	P_W	P_TURN	XX
4	Thomas R	175	5	175
2	Will J	200	4	375
5	George W	250	1	625
3	John A	350	2	975
1	James E	500	6	1475
6	Thomas J	(null)	3	1475

200 + 175

250 + 200 + 175

If you see the above example, what is happening here. till weight 975 will go in the elevator. So, **John A** is the last person.

We have a table named Customer with columns id, name, visited_on and amount. We want to calculate a 7-day moving average of the amount for each date.

```
SELECT c.*,  
AVG(amount) OVER (ORDER BY visited_on ROWS BETWEEN 6 PRECEDING AND current row) AS moving_avg  
FROM  
Customer c;
```

ROWS BETWEEN 6 PRECEDING AND current row : last 6 rows and current row : 7 days and took average

customer_id	name	visited_on	amount	moving_avg
1	Jhon	2019-01-01	100	100.00
2	Daniel	2019-01-02	110	105.00
3	Jade	2019-01-03	120	110.00
4	Khaled	2019-01-04	130	115.00
5	Winston	2019-01-05	110	114.00
6	Elvis	2019-01-06	140	118.33
7	Anna	2019-01-07	150	122.86
8	Maria	2019-01-08	80	120.00
9	Jaze	2019-01-09	110	120.00
1	Jhon	2019-01-10	130	121.43
3	Jade	2019-01-10	150	124.29

we want
to
display
moving
average

=====

Running Total (Cumulative Sum): Suppose you have a sales table with columns order_date, revenue, and product_id. To calculate the cumulative revenue for each product, you can use the following query:

```
SELECT  
product_id,  
order_date,  
revenue,  
SUM(revenue) OVER (PARTITION BY product_id ORDER BY order_date) AS cumulative_revenue  
FROM sales;
```

=====